Lab 11 — Hands-on lab: first contact with the cluster

Goal: install k3s in your WSL, explore the cluster with `kubectl`, start a first Pod, bring in the `api:1.0` image built with Podman, then see the reconciliation loop with your own eyes — via the `podman generate kube` bridge.

Prerequisites — Docker part completed (labs 00 to 10). Image `api:1.0` present in Podman, `~/labo-docker/07/nginx-pod.conf` present. Ubuntu/WSL with `systemd=true`, `sudo`, and at least 4 GB of RAM for WSL. The output below comes from k3s v1.34.1; the node name is your machine's (here `wsl-ubuntu`).

Files provided (`files/`) - `pod-api.yaml` — a minimal Pod for the API of the running example.

Step 0 — Installing k3s (WSL only)

```
curl -sfL https://get.k3s.io | sudo sh -s - --write-kubeconfig-mode 644
sudo systemctl status k3s --no-pager | head -3
which kubectl ; ls -l $(which kubectl)
```

Observe the script downloading the binary, creating the `k3s.service` service and starting it (`active (running)`), and a `kubectl` that is nothing but a link to `/usr/local/bin/k3s`. This step cannot be replayed in the sandbox this lab was written in: the output is described, not quoted. Allow a minute before the node is ready.

Step 1 — Getting to know the cluster

```
kubectl get nodes
kubectl get pods -A
kubectl cluster-info | head -2
ls -l /etc/rancher/k3s/k3s.yaml ; grep -E 'server:' /etc/rancher/k3s/k3s.yaml
kubectl config get-contexts
```

Observe one node `wsl-ubuntu Ready control-plane … v1.34.1+k3s1`; in `kube-system`, the components shipped by k3s: `coredns`, `local-path-provisioner`, `metrics-server`, `traefik`, `svclb-traefik-…`, and two `helm-install-traefik…` Pods in `Completed`; the API server at `https://127.0.0.1:6443`; a kubeconfig in `-rw-r--r--`; and a `default` context with no namespace.

Explanation. A single node plays both control plane and worker. Everything you will see goes through the API server from `cluster-info`.

```
kubectl api-resources | head -8
kubectl explain pod.spec.containers | head -12
```

Observe the table of object types (`configmaps`, `events`, `namespaces`…, with their short names and whether they are `NAMESPACED`) and the built-in documentation of a Pod's `containers` field.

Step 2 — A namespace for the lab

```
kubectl create namespace lab11
kubectl config set-context --current --namespace=lab11
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}' ; echo
```

Observe `namespace/lab11 created`, `Context "default" modified.`, then `lab11`: every following command works in that namespace with no `-n`.

Step 3 — A first Pod, with no file

```
kubectl run web --image=docker.io/library/nginx:alpine
kubectl get pods
sleep 8 ; kubectl get pods -o wide
kubectl describe pod web | sed -n '/^Events/,$p'
```

Observe `pod/web created`, a `ContainerCreating` state then `Running` with a `10.42.0.x` IP on the `wsl-ubuntu` node, and the Events timeline: `Scheduled` (by `default-scheduler`), `Pulling`, `Pulled … in 3.75s`, `Created`, `Started` (by `kubelet`).

```
kubectl logs web | tail -2
kubectl exec web -- nginx -v
kubectl port-forward pod/web 18080:80 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080 | head -4 ; kill %1
kubectl delete pod web
```

Observe nginx's logs, `nginx version: nginx/1.x`, the welcome page through the `port-forward` tunnel (no published port: `kubectl` relays), then `pod "web" deleted from lab11 namespace`.

Explanation. `logs`, `exec`, `delete` are the gestures of lab 03; `describe` and its Events are new: the cluster records every step of every component.

Step 4 — The Podman bridge

```
mkdir -p ~/labo-docker/11 && cd ~/labo-docker/11
podman pod create --name demo -p 18090:80
podman run -d --pod demo --name demo-api api:1.0
podman run -d --pod demo --name demo-web -v ~/labo-docker/07/nginx-
pod.conf:/etc/nginx/conf.d/default.conf:ro nginx:alpine
sleep 2 ; curl -s localhost:18090/api/actuator/health ; echo
podman generate kube demo > demo-pod.yaml
grep -E 'kind:|name:|image:|hostPort|hostPath' demo-pod.yaml
podman pod rm -f -t 0 demo
podman kube play demo-pod.yaml | tail -3
sleep 3 ; curl -s localhost:18090/api/actuator/health ; echo
podman kube down demo-pod.yaml | tail -1 ; podman pod ps
```

Observe `{"status":"UP"}` from the Podman pod; in the generated YAML, `kind: Pod`, the two `image:` lines, a `hostPort: 18090` and a `hostPath` volume pointing to your nginx file; after `kube play`, the pod recreated **from the YAML** (`demo-demo-api`, `demo-demo-web`) answering again; then no pod left after `kube down`.

Explanation. Podman writes and reads Kubernetes YAML. That file describes exactly your pod — but `hostPath` and `hostPort` tie it to your machine; a cluster will replace them with a ConfigMap and a Service.

Step 5 — The missing image, then the import

```
kubectl apply -f <path-to-the-lab>/files/pod-api.yaml
sleep 6 ; kubectl get pods
kubectl describe pod api | sed -n '/^Events/,$p' | tail -4
```

Observe `pod/api created`, then `ErrImagePull` or `ImagePullBackOff`, with in the Events `Failed to pull image "localhost/api:1.0": … failed to pull and unpack image` and `Back-off pulling image`.

```
podman save localhost/api:1.0 | sudo k3s ctr images import -
sudo k3s crictl images | grep -E 'api|nginx'
sleep 20 ; kubectl get pods -o wide
```

Observe `Importing … done` on the import side; in `crictl images`, `localhost/api 1.0` next to `docker.io/library/nginx alpine` (pulled by the cluster in step 3); then, without restarting anything, `api 1/1 Running` with its IP.

Explanation. The cluster has its own image store. The kubelet, which kept retrying, found the image after the import — `imagePullPolicy: IfNotPresent` tells it to use what is present.

```
kubectl get pod api -o yaml | grep -E '^  phase:|podIP:|hostIP:'
kubectl exec api -- wget -qO- http://localhost:8080/ ; echo
kubectl port-forward pod/api 18080:8080 > /dev/null 2>&1 &
sleep 2 ; curl -s localhost:18080/actuator/health ; echo ; kill %1
sudo k3s crictl ps | grep -E 'api|CONTAINER' | cut -c1-100
```

Observe `phase: Running`, a `podIP` in `10.42.0.x` and a `hostIP` (your WSL's); `{"message":"bonjour depuis kubernetes","profile":"default"}` (the variable from the manifest); `{"status":"UP"}` through the tunnel; and in `crictl ps` an `api` container — the one `podman ps` does not show.

Step 6 — The reconciliation loop

```
kubectl delete pod api
kubectl get pods
kubectl create deployment api --image=localhost/api:1.0
sleep 6 ; kubectl get deployment,replicaset,pods
```

Observe the lone Pod deleted for good (`No resources found`), then three objects: `deployment.apps/api 1/1`, `replicaset.apps/api-5f68dd6795 1 1 1`, and a Pod `api-5f68dd6795-xxxxx Running`.

```
POD=$(kubectl get pods -l app=api -o jsonpath='{.items[0].metadata.name}')
kubectl delete pod $POD
sleep 6 ; kubectl get pods
kubectl get deployment api -o jsonpath='{.spec.replicas} {.status.readyReplicas}' ; echo
```

Observe `pod "api-5f68dd6795-xxxxx" deleted`, and right away a **new** Pod `api-5f68dd6795-yyyyy Running`; `1 1`: one replica desired, one replica ready.

Explanation. You deleted the Pod; the ReplicaSet noticed the gap and created another one, without you asking for anything. That is the declarative model at work — and the subject of lab 12.

Clean-up

```
kubectl delete namespace lab11
kubectl config set-context --current --namespace=default
kubectl get pods -A | grep -c lab11
```

Observe `namespace "lab11" deleted` (with everything it held: Deployment, ReplicaSet, Pod), the context back on `default`, and `0`. The `localhost/api:1.0` image stays in the cluster; the following labs use it. To stop k3s between sessions: `sudo systemctl stop k3s`; to remove everything one day: `/usr/local/bin/k3s-uninstall.sh`.

What you must be able to state now

- One `Ready` node, six components in `kube-system`, an API server on `127.0.0.1:6443`, a kubeconfig in `644`.
- `kubectl run` → `Scheduled`, `Pulling`, `Pulled`, `Created`, `Started`; `logs`, `exec`, `port-forward`, `delete` as with Podman.
- `podman generate kube` writes a `kind: Pod` with `hostPort` and `hostPath`; `podman kube play` replays it.
- Podman image ≠ cluster image: `ImagePullBackOff`, then `podman save | sudo k3s ctr images import -`, and `Running` without restarting anything.
- A lone Pod deleted does not come back; under a Deployment, a new Pod appears within seconds: `1 1`.
- `kubectl delete namespace lab11` takes away everything the lab created.